

RESCUE AI: Radio Enhancement & Speech Clarification for Urgent Emergencies

Next-Generation Offline-First Audio Edge Intelligence Platform for Disaster Recovery

Project Team - VANGUARD

Technical Track: Intelligent Signal Processing and Noise Detection

System Version 1.0.4 – Compiled: June 18, 2026

Abstract

During critical emergency incidents, communications infrastructure is frequently degraded. First responders rely on tactical radio frequencies saturated with harsh acoustic interference (e.g., sirens, wind, and structural collapse rumblings). Legacy Automatic Speech Recognition (ASR) setups fail entirely without localized internet connectivity. This paper introduces **RESCUE AI**, an end-to-end, off-grid capable edge platform that cleans raw radio audio streams, attributes conversational turns to specific personnel, and extracts structured emergency analytics using local deep learning models in under one second. The platform operates under an absolute data privacy paradigm and features client-side failover mechanics to guarantee structural reliability when underlying host systems degrade.

1 Introduction & Project Objectives

In emergency dispatch, response efficiency dictates trauma survival rates. The initial 60 minutes of an event—the “Golden Hour”—are highly critical; a 60-second dispatch latency reduction correlates directly with up to a 10% increase in patient survivability. However, extreme acoustic environments often increase transcription verification times by 20–35%.

RESCUE AI addresses these failures through four core operational objectives:

1. **Offline Autonomy:** Operate with a 0.0% cloud computing dependency to survive natural disasters and local mast outages.
2. **Mathematical Denoising Pre-processing:** Isolate human voice prints using mathematical difference equations before running neural inference, preventing hallucination errors.
3. **Vocal Blueprint Classification:** Separate overlapping unit responses via multi-dimensional speaker embeddings.
4. **Cognitive Action Profiling:** Transform raw, garbled speech streams into immediate, structured JSON metrics (hazards, locations, and checklists).

2 System Architecture

The backend is structured around a decoupled **React 18** dashboard interface and a **FastAPI (Python)** processing pipeline. The data flow moves sequentially through ingestion, pre-filtering, neural extraction, and structured synthesis.

3 Mathematical & Engineering Formulations

RESCUE AI implements robust deterministic digital signal processing algorithms to clean audio prior to ingestion by deep learning layers.

Table 1: System Capabilities and Comparative Benchmarks

Metric / Capability	Legacy Radio Systems	Cloud ASR Stacks	RESCUE AI (Proposed)
Network Dependence	100% Voice Radio Relays	100% High-Bandwidth API	0.0% Offline Standalone
Noise Resilience	Manual Operator Strain	High Word Error Failures	DSP-Sanitized Inputs
Voice Attribution	Subjective/Visual Calls	Unreliable Post-clustering	Neural Speaker Diarization
Data Custody	Vulnerable Open Lines	Third-Party Server Storage	Absolute Local Sandboxing
System Failover	Complete Silence on Cut	Application Crashes	Browser-Fallback DSP & ASR

3.1 Butterworth 4th-Order Bandpass Filter

Human speech energy is mathematically isolated between 300 Hz and 3400 Hz. Low-frequency background rumbles and high-frequency structural breakages are clipped using an N -order Butterworth polynomial filter, designed with a flat frequency response in the passband:

$$|H(\omega)|^2 = \frac{1}{1 + \left(\frac{\omega}{\omega_c}\right)^{2N}} \quad (1)$$

Where $N = 4$ provides a steep roll-off to attenuate out-of-band noise without introducing disruptive phase delay. In the time domain, this is evaluated dynamically via recursive difference equations:

$$y[n] = \sum_{k=0}^P b_k x[n-k] - \sum_{m=1}^Q a_m y[n-m] \quad (2)$$

3.2 Statistical Spectral Subtraction

Assuming raw environmental noise $n(t)$ is additive and uncorrelated with the target speech signal $x(t)$, the noisy input waveform $y(t) = x(t) + n(t)$ is mapped to the frequency domain via Short-Time Fourier Transform (STFT). Using a captured noise power spectrum estimation $E[|N(\omega)|^2]$ during quiet windows, the denoised spectrum $\hat{X}(\omega)$ is derived:

$$\hat{X}(\omega) = (|Y(\omega)|^2 - \alpha \cdot E[|N(\omega)|^2])^{1/2} \quad (3)$$

Where α represents an over-subtraction factor ($1.0 \leq \alpha \leq 3.0$) tuned to suppress musical noise artifacts and persistent background humming before executing an Inverse STFT.

3.3 Temporal Intersection Alignment

Transcription slices generated by the faster-whisper engine must map correctly to the speaker turn periods outputted by pyannote.audio. For a transcript segment $T = [t_{\text{start}}, t_{\text{end}}]$ and a speaker slice $S = [s_{\text{start}}, s_{\text{end}}]$, the overlap duration D is calculated as:

$$D(T, S) = \max(0, \min(t_{\text{end}}, s_{\text{end}}) - \max(t_{\text{start}}, s_{\text{start}})) \quad (4)$$

The transcript is assigned to the corresponding speaker profile S_i utilizing a maximum intersection policy:

$$\text{Assigned Speaker} = \arg \max_{S_i} D(T, S_i) \quad (5)$$

4 Technical Deployment Guide

The unified engine and client application are brought online with the following standard bootstrap commands:

```
# Clone project and configure local virtual environment
git clone https://github.com/sourya2007/rescue-ai.git
cd rescue-ai && python -m venv .venv
source .venv/bin/activate # Use .venv\Scripts\activate on Windows

# Install Python ML/DSP runtimes and Node dependencies
pip install -r requirements.txt && npm install

# Download Llama 3.2:3b local weights and execute
ollama pull llama3.2:3b
export HF_TOKEN="your_huggingface_access_token_here"
python backend.py
```

Listing 1: System Bootstrap Script

5 Graceful Fallback Mechanics

In the event of severe CPU/GPU host hardware failure, the system falls back entirely to browser-based utilities:

- **Client-Side Fallback DSP:** Initiates native WebAudio API context instances (`audioDsp.ts`) using biquad and analysis nodes to filter raw browser input channels.
- **Client-Side Local NLP:** Leverages optimized keyword trees and regex parsing rulesets (`localEmergency`) to extract immediate, low-latency target locations, incident tags, and critical alert warnings.

6 Conclusion & Future Work

RESCUE AI demonstrates that offline disaster response can achieve near zero-latency transcription and contextual parsing on standard, low-power laptop computers. Future development cycles will focus on compiling the core pipeline to run on small ARM-based field radios and microcomputers, enabling lightweight tactical audio systems to operate completely autonomously in remote zones.